SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: CSA04 COURSE CODE: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments. (BL2, BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Total Marks:25**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I S.Hasini [192525294] *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

Question 1 **10 Marks**

A university computer laboratory allows multiple students to execute programs simultaneously on a shared Linux server.

(a) Identify the type of Operating System suitable for this environment and justify your answer.

(b) State any two responsibilities of the kernel in managing multiple user processes.

(c) Write a C-based system call sequence (or pseudo-code) to create a child process and display the Process IDs of both the parent and child.

Question 2 **10 Marks**

An application needs to store and retrieve student records from a text file.

(a) List the system calls required for file creation, writing, reading, and closing.

(b) Explain the purpose of any two file management system calls.

(c) Write a C-based system call sequence (or pseudo-code) to create a file, write a student name, and close the file.

Question 3 **10 Marks**

A user wants to copy the contents of one file to another.

(a) List the system calls required to complete this operation.

(b) Explain the purpose of any two of these system calls.

(c) Write a pseudo-code or C-based system call sequence to copy the contents of one file to another.

Question 4 **10 Marks**

A program needs to execute another program after creating a child process.

(a) State the purpose of the fork() and exec() system calls.

(b) Explain the role of the wait() system call.

(c) Write a system call sequence using fork(), exec(), and wait().

Question 5 **10 Marks**

An operating system manages multiple files stored on secondary memory.

(a) Explain the importance of file management in an operating system.

(b) State the purpose of the open() and close() system calls.

(c) Write a C-based system call sequence to open a file, read its contents, and close it.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	35%	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25%	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15%	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15%	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10%	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Marks Evaluation:

Question No.	Conceptual Understanding (35%)	Accuracy of Answers (25%)	Application of Knowledge (15%)	Completeness of Response (15%)	Clarity & Presentation (10%)	Total Marks (100%)
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

1) a. Multi-user Operating System

It is suitable for this environment. It allows multiple users to access and use the same computer system simultaneously.

Advantages:

1. It allows multiple students to use the server at the same time.
2. It manages CPU time among different users and processes.
3. It provides memory protection between programs.
4. It manages shared resource efficiently

b) Responsibilities of the kernal

1. Process management:

* Create and terminate processes
* Schedules the CPU time to different processes
* Manages the process states and context switching.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
int main()
{
    pid_t pid;
    pid = fork();
    if (pid < 0)
    {
```

```
        printf("Process creation failed\n");
        }
        else if (pid == 0)
        {
            printf ("child process ID: %d \n", getpid());
            printf ("parent Process ID: %d \n", getpid());
        }
        else
        {
            printf("Parent process ID: %d\n", getpid());
            printf(" child process ID: %d \n", pid);
        }
        return 0;
    }
```

Output :

Parent Process ID: 1200

child process ID: 1201

child process ID: 1201

Parent Process ID: 1200

2
a) System calls used in file management

1. open() - opens an existing file or create a new file with suitable flags.

2. Creat() - Creates a file or truncates an existing file.

3. Write() - write data into the file.

The open() system call opens a file for reading or writing. It returns a file descriptor on success.

```
int fd = open("student.txt", O_WRONLY | O_CREATE, 0644);
```

* student.txt specifies the filename
* O_WRONLY opens the file for writing.
* O_CREATE creates the file if it does not exist.
* 0644 specifies file permissions.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    char name[] = "Hasini";

    fd = open("student.txt", O_WRONLY | O_CREATE | O_TRUNC, 0644);

        if (fd == -1)
        {
            printf("File creation failed\n");
```

```
    }
    write (fd, name, sizeof(name) - 1);

    close(fd);
    printf("student name written successfully\n");
    return 0;
}
```

3) a) System Calls Required to copy a File

1. open() - open the source and destination files.
2. read() - Reads data from the source file.
3. write() - write data into the destination file.
4. close() - closes both files after copying.

write() system call

The write() system call writes data from a memory buffer into an open file.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
int main()
{
    int source, destination, n;
    char buffer[100];
    source = open("source.txt")
    if (source == -1)
```

```
        printf ("Cannot open destination file\n");
        close (source);
        return 1;
    }
    while ((n = read (source, buffer, size (buffer))) > 0)
    {
    write (destination, buffer, n);
    }
    close (source);
    close (destination);
    printf ("File copied successfully \n");
    return 0;
}
```

4) a) Purpose of fork() and exec() system calls

1. fork() System call

The fork() system call creates a new process called a child process by duplicating the calling process

Purposes:

* create a child process.
* Allows parent and child processes to execute independently
* Returns 0 to the child process
* Returns the child's PID to the parent process.

The wait() system call allow a parent process to wait until one of its child processes terminates.

1. Suspends the parent process until a child terminates or the specified wait condition is met.
2. Allow the parent to collect the child's termination status.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main()
{
  pid_t pid;
  pid = fork();
  if (pid<0)
  {
    printf("Fork failed \n");
    return 1;
  }
  else if (pid ==0)
   {
    printf("child process executing\n");
    execlp("ls", "ls", "-l", (char*) Null);
    printf("Exec failed\n");
    return 1;
}
```

5) Importance of File management in an operating system.

File management is an important function of an operating systems that organizes, stores, retrieves, and protects file on secondary storage devices.

File organization: organizes files into directories and folders for easy access.

File creation and Deletion: Allows users to create, rename and delete file.

Releases the file descriptor

* Ends the program's current access through that descriptor
* Helps the free operating systems resources associated with open file.

```
#include <stdio.h>
#include <fcntl.h>
#include<unistd.h>
int main()
{
  int fd, n;
  char buffer [100];
  fd = open("student.txt");
  if (fd==-1)
  {
  n=read (fd, buffer, sizeof (buffer) - 1);
```

```
    close(fd);
    return 1;
  }
  buffer[n] = '\0';
  printf("File contents : %s\n", buffer);
  close(fd);
  return 0;
}
```